

Trabalho IA 3: IA Generativa

Tema 2: *Output* Estruturado

Brunna Moura Carlos Cruz Rafael Queiroz Victor Bitencourt

Universidade do Estado da Bahia (UNEB)

4 de Julho de 2026

Problema atacado

- O problema consiste na criação de uma aplicação RAG que consiga retornar *output* estruturado, com mecanismos de validação, mostrando que uma solução de IA pode ser utilizada em sistemas maiores, integrada a fluxos existentes.
- Principal mecanismo utilizado é o *output* estruturado, fornecido nativamente por LLMs hoje em dia.
- Como funciona o *output* estruturado nativo de LLMs?

Base Documental

Documento Fonte

Coloque uma imagem da capa do documento aqui!

Base Documental

Informações

- A base documental consiste no relatório da CPMI de 8 de janeiro de 2023, quando houve tentativa de golpe de Estado no Brasil.
- A base documental foi escolhida estrategicamente, tratando de um assunto possivelmente conhecido por modelos de IA, no entanto, esperando-se no presente trabalho que a mesma traga fontes para sustentar seus resultados, fornecendo uma análise crítica do desempenho do modelo de LLM no fornecimento confiável e embasado de informações.

Arquitetura da Solução

Bibliotecas utilizadas

Coloque uma imagem das bibliotecas utilizadas aqui!

Arquitetura da Solução

Explicação sobre as bibliotecas

- Carregamento de arquivos: `pymupdf` - um dos melhores carregadores de PDF do mercado, se destacando para PDFs com representação XML presente (*tagged PDFs*). Pela integridade do documento fonte, a biblioteca cumpriu bem sua tarefa. Contras: licença estritamente AGPL.
- Segmentação de *chunks*: `langchain-text-splitters`, utilizando o `RecursiveCharacterTextSplitter`. Chunks de tamanho 2000 com *overlap* de 200.

Arquitetura da Solução

Explicação sobre as bibliotecas

- *Embeddings*: modelo local de redes neurais convolucionais `pt_core_news_lg`, fornecido pela biblioteca `spaCy`, pela leveza em processamento local e dispensa de limites de API decorrente. Evidentemente, o mesmo modelo para *embedding* dos *chunks* é o mesmo modelo utilizado para *embedding* das perguntas do usuário, que é o correto a se fazer.
- Modelo de LLM: `gemini-3.1-flash-lite`, melhor modelo “econômico” da plataforma Gemini API hoje para tarefas simples e diretas como RAG de pergunta-e-resposta. Conecta-se ao modelo via biblioteca `langchain-google-genai`.

Pipeline de Ingestão

- A pipeline de ingestão é feita ao executar o módulo `src/run_ingestion.py`. É feito:
 - o carregamento do text do documento PDF,
 - segmentação dos *chunks*,
 - o *embedding* do seu conteúdo,
 - e a inserção das passagens com seus *embeddings* associados dentro do banco PostgreSQL.

Pipeline de Ingestão

Diagrama

Coloque um diagrama Graphviz do fluxo de execução aqui!

Recuperação Vetorial: Armazenamento

Disposição do banco

O banco é otimizado, ao utilizar-se um esquema que utiliza quantização escalar com o tipo `halfvec` para tornar a ingestão mais rápida e eficiente, com perda mínima de precisão, tornando o desenvolvimento mais rápido sem sacrificar os resultados.

Script

```
CREATE TABLE chunk (  
    id INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    snippet TEXT NOT NULL,  
    embedding halfvec(300) NOT NULL,  
    page INTEGER NOT NULL  
);
```

Recuperação Vetorial: Busca

Descrição

- Para a busca semântica, uma similaridade de cosseno comum é aplicada, utilizando os operadores usuais fornecidos pela extensão PGVector.
- A busca semântica compara o embedding da pergunta do usuário com o que está no banco e retorna as passagens relevantes, de acordo com um valor de top-k correspondente. Foi estabelecido um valor de $k=5$ para os testes.

Consulta SQL

```
SELECT snippet, page, embedding <=> %s::halfvec AS distance
FROM chunk
ORDER BY distance
LIMIT %s;
```

Recuperação Vetorial: Pergunta

O formato do prompt é montado manualmente, com a pergunta do usuário no final. Foi feito dessa forma por ser uma boa prática: prompts no final com partes fixas no início se beneficiam do *prompt caching* fornecido pelas APIs. A seguir, o template de prompt montado pela equipe, em um formato XML, que costuma ser o formato mais bem entendido pelas LLMs, dado que delimita os limites de cada seção com tags, o que é satisfatório para a forma como a LLM processa texto: de forma linear.

Recuperação Vetorial: Pergunta

<system>

O contexto a seguir vem de busca semântica.
Responda o usuário com base no contexto retornado.
Há a possibilidade do contexto não ser relevante,
dado que vem de uma busca semântica direta.

</system>

<context>

{"\n\n".join(search_results)}

</context>

<user>

{query}

</user>

Validação ou Mecanismo Híbrido

A validação do objeto retornado já é feita internamente pelo framework langchain, utilizado no trabalho. No presente trabalho, a implementação de validação é feita no caso de erros de solicitação à API do modelo. Em termos qualitativos, o formato do objeto a ser retornado para aplicação, correspondente à resposta da IA com as fontes, que consiste no output estruturado proposto no trabalho, aceita nenhuma fonte como resultado. Essa é a forma com que a aplicação lida com respostas possivelmente não encontradas. A seguir, o formato de objeto Pydantic esperado que o LLM retorne.

```
from pydantic import BaseModel, Field

class AIAnswer(BaseModel):
    content: str = Field(description="Sua resposta")
    sources: str | None = Field(description="Todas as fontes para embasar a resposta")
```

Resultados Experimentais

Experimento: acertos x total

Categoria	Acertos	Total
casos fáceis	3	6
casos médios	4	7
casos ambíguos	3	6
casos onde a base de conhecimento é insuficiente	5	6
casos que testam os limites da aplicacao	5	5
Total	20	30

Principais Falhas

- Embeddings ruins
- Continua...

Considerações sobre uso de IA

- (Só copiar o que já anotei)

Dúvidas?

